

# SQLD 57회 기출 특강

## 유형1. COUNT

- NULL은 COUNT 하지 않는다. (AVG, SUM, MIN, MAX도 마찬가지)
- 조건에 맞는 행이 없는 경우 COUNT 결과는 0이다.
- 조건에 맞는 행이 없는 경우 SUM, AVG, MIN, MAX 결과는 NULL이다.
- COUNT(DISTINCT 컬럼1) : 컬럼1의 값에 대한 UNIQUE 행의 수 리턴
  - ※ COUNT(DISTINCT \*) 불가
  - ※ COUNT(DISTINCT 컬럼1, 컬럼2) : 컬럼1, 컬럼2의 값에 대한 UNIQUE 행의 수 리턴

문제1. 다음 중 SQL 결과가 다른 것은? (단, DEPTNO 컬럼에는 NULL 값이 존재하지 않는다.)

- ① SELECT COUNT(DISTINCT DEPTNO) AS CNT FROM EMP;
- ② SELECT COUNT(\*) AS CNT FROM (SELECT DISTINCT DEPTNO FROM EMP);
- ③ SELECT SUM(1) AS CNT FROM (SELECT DISTINCT DEPTNO FROM EMP);
- ④ SELECT DISTINCT COUNT(DEPTNO) AS CNT FROM EMP;

문제2. 다음 SQL 실행 결과로 알맞은 것은?

SELECT NVL(COUNT(\*),'9999') FROM EMP WHERE 1=2;

- ① NULL
- ② 0
- ③ 9999
- ④ 오류

문제3. 다음 SQL 실행 결과로 알맞은 것은?

<EMP TABLE>

| EMPNO | ENAME  | JOB       | MGR  | HIREDATE            | SAL  | COMM | DEPTNO |
|-------|--------|-----------|------|---------------------|------|------|--------|
| 7369  | SMITH  | CLERK     | 7902 | 1980/12/17 00:00:00 | 800  |      | 20     |
| 7499  | ALLEN  | SALESMAN  | 7698 | 1981/02/20 00:00:00 | 1600 | 300  | 30     |
| 7521  | WARD   | SALESMAN  | 7698 | 1981/02/22 00:00:00 | 1250 | 500  | 30     |
| 7566  | JONES  | MANAGER   | 7839 | 1981/04/02 00:00:00 | 2975 |      | 20     |
| 7654  | MARTIN | SALESMAN  | 7698 | 1981/09/28 00:00:00 | 1250 | 1400 | 30     |
| 7698  | BLAKE  | MANAGER   | 7839 | 1981/05/01 00:00:00 | 2850 |      | 30     |
| 7782  | CLARK  | MANAGER   | 7839 | 1981/06/09 00:00:00 | 2450 |      | 10     |
| 7788  | SCOTT  | ANALYST   | 7566 | 1987/04/19 00:00:00 | 3000 |      | 20     |
| 7839  | KING   | PRESIDENT |      | 1981/11/17 00:00:00 | 5000 |      | 10     |
| 7844  | TURNER | SALESMAN  | 7698 | 1981/09/08 00:00:00 | 1500 | 0    | 30     |
| 7876  | ADAMS  | CLERK     | 7788 | 1987/05/23 00:00:00 | 1100 |      | 20     |
| 7900  | JAMES  | CLERK     | 7698 | 1981/12/03 00:00:00 | 950  |      | 30     |
| 7902  | FORD   | ANALYST   | 7566 | 1981/12/03 00:00:00 | 3000 |      | 20     |
| 7934  | MILLER | CLERK     | 7782 | 1982/01/23 00:00:00 | 1300 |      | 10     |

<SALGRADE TALE>

| GRADE | LOSAL | HISAL |
|-------|-------|-------|
| 1     | 700   | 1200  |
| 2     | 1201  | 1400  |
| 3     | 1401  | 2000  |
| 4     | 2001  | 3000  |
| 5     | 3001  | 9999  |

SELECT COUNT(CASE WHEN S.GRADE = 2 THEN 2 END) AS CNT  
FROM EMP E, SALGRADE S  
WHERE E.SAL BETWEEN S.LOSAL AND S.HISAL;

- ① 1
- ② 2
- ③ 3
- ④ 4

## 유형2. 날짜 반올림/버림

- 날짜는 ROUND, TRUNC로 각각 단위별 반올림, 버림 처리 가능
- 반올림, 버림을 할 위치는 YEAR, MONTH 전달 가능(생략 시 시간 단위에서 반올림/버림 진행)
  - ※ YEAR : MONTH 단위에서 반올림/버림 진행하여 YEAR를 조절
  - ※ MONTH: DAY단위에서 반올림/버림 진행하여 MONTH를 조절

문제1. 다음 SQL 실행 결과로 알맞은 것은?

(단, 오늘날짜는 2025/01/01 13:56:00 이라고 가정)

```
SELECT TRUNC(SYSDATE) AS DATE1,  
       ROUND(SYSDATE) AS DATE2  
FROM DUAL;
```

|   | DATE1               | DATE2               |
|---|---------------------|---------------------|
| ① | 2025/01/01 00:00:00 | 2025/01/01 00:00:00 |
| ② | 2025/01/01 00:00:00 | 2025/01/02 00:00:00 |
| ③ | 2025/01/01 00:00:00 | 2025/01/02 13:56:00 |
| ④ | 2025/01/01 13:56:00 | 2025/01/02 13:56:00 |

문제2. 다음 SQL 실행 결과로 알맞은 것은?

(단, 오늘날짜는 2025/01/16 13:56:00 이라고 가정)

```
SELECT TRUNC(SYSDATE, 'MONTH') AS DATE1,  
       ROUND(SYSDATE, 'MONTH') AS DATE2  
FROM DUAL;
```

|   | DATE1               | DATE2               |
|---|---------------------|---------------------|
| ① | 2025/01/01 00:00:00 | 2025/01/01 00:00:00 |
| ② | 2025/01/16 00:00:00 | 2025/01/16 00:00:00 |
| ③ | 2025/01/01 00:00:00 | 2025/02/01 00:00:00 |
| ④ | 2025/02/01 00:00:00 | 2025/02/01 00:00:00 |

### 유형3. CHAR 타입과 VARCHAR 타입 비교

- VARCHAR2(N) : 최대 N Bytes 문자 입력 가능(뒤에 공백 삽입 없음)
- CHAR(N) : 최대 N Bytes 문자로 만든 뒤 입력 됨(뒤에 공백 삽입)
- 앞자리 공백은 공백까지 일치해야 동일한 문자임
- 뒷자리 공백은
  - 1) CHAR 타입 : 뒷자리 공백 제거 후 비교
  - 2) VARCHAR2 타입 : 뒷자리 공백 제거하지 않고 그대로 비교

문제1. 다음 SQL 실행 결과로 적절한 것은?

```
CREATE TABLE CHAR_TAB1(COL1 CHAR(4), COL2 VARCHAR2(4));
INSERT INTO CHAR_TAB1 VALUES ('AA','AA');
INSERT INTO CHAR_TAB1 VALUES ('BB','BB ');
INSERT INTO CHAR_TAB1 VALUES ('CC',' CC');
INSERT INTO CHAR_TAB1 VALUES (' D',' D ');
SELECT COUNT(*) FROM CHAR_TAB1 WHERE COL1 = COL2;
```

- ① 1
- ② 2
- ③ 3
- ④ 4

문제2. 다음 SQL 실행 결과로 적절한 것은?

```
CREATE TABLE CHAR_TAB2(COL1 VARCHAR2(4), COL2 VARCHAR2(4));
INSERT INTO CHAR_TAB2 VALUES ('AA','AA');
INSERT INTO CHAR_TAB2 VALUES ('BB','BB ');
INSERT INTO CHAR_TAB2 VALUES ('CC',' CC');
INSERT INTO CHAR_TAB2 VALUES (' D',' D ');
COMMIT;
SELECT COUNT(*) FROM CHAR_TAB2 WHERE COL1 = COL2;
```

- ① 1
- ② 2
- ③ 3
- ④ 4

문제3. 다음 SQL 실행 결과로 적절한 것은?

```
CREATE TABLE CHAR_TAB3(COL1 CHAR(4), COL2 CHAR(4));
```

```
INSERT INTO CHAR_TAB3 VALUES ('AA','AA');
```

```
INSERT INTO CHAR_TAB3 VALUES ('BB','BB ');
```

```
INSERT INTO CHAR_TAB3 VALUES ('CC',' CC');
```

```
INSERT INTO CHAR_TAB3 VALUES (' D',' D ');
```

```
INSERT INTO CHAR_TAB3 VALUES ('E ','E');
```

```
SELECT COUNT(*) FROM CHAR_TAB3 WHERE COL1 = COL2;
```

① 1

② 2

③ 3

④ 4

## 유형4. SUBSTR, INSTR

- SUBSTR(대상, m, n) 에서 n 생략 시 끝까지 추출

※ CHAR 타입의 컬럼인 경우 뒤에 공백이 있을 수 있으므로 공백까지 추출될 수 있음

※ m은 음수 전달 가능(뒤에서 m번째 위치를 의미)

※ n은 음수 전달 불가

문제1. 다음 SQL 실행 결과 중 다른 하나는? (단, COL1은 CHAR(9) 타입이다.)

| COL1     |
|----------|
| ABCDEFGH |

- ① SELECT TRIM(SUBSTR(COL1, 7, 2)) AS RESULT FROM DUAL;
- ② SELECT TRIM(SUBSTR(COL1, -2)) AS RESULT FROM DUAL;
- ③ SELECT TRIM(SUBSTR(COL1, 7)) AS RESULT FROM DUAL;
- ④ SELECT TRIM(SUBSTR(COL1, INSTR(COL1,'G'), 2)) AS RESULT FROM DUAL;

문제2. 다음 결과를 얻기 위한 빈칸으로 적절한 것은?

| TEL          |
|--------------|
| 031)345-3948 |
| 02)4534-3928 |
| 055)938-0093 |
| 02)392-1938  |

| RESULT |
|--------|
| 031    |
| 02     |
| 055    |
| 02     |

- ① SELECT SUBSTR(TEL, 1, 2) AS RESULT FROM TAB;
- ② SELECT SUBSTR(TEL, 1, 3) AS RESULT FROM TAB;
- ③ SELECT SUBSTR(TEL, INSTR(TEL, ')'), 3) AS RESULT FROM TAB;
- ④ SELECT SUBSTR(TEL, 1, INSTR(TEL, ')')-1) AS RESULT FROM TAB;

## 유형5. NOT IN / NOT EXISTS

- 일반적으로 NOT IN과 NOT EXISTS의 결과는 같다.
- NOT IN은 서브쿼리 결과에 NULL이 하나라도 있으면, 비교 자체가 불가능해져서 아무것도 출력되지 않는다.

문제1. 다음 SQL 중 실행 결과가 다른 하나는?  
(단, TAB1과 TAB2의 ID 컬럼에는 NULL이 없다)

- ① SELECT COUNT(\*) FROM TAB1  
WHERE NOT EXISTS (SELECT 'X' FROM TAB2 WHERE TAB1.ID = TAB2.ID);
- ② SELECT COUNT(\*) FROM TAB1  
WHERE ID NOT IN (SELECT ID FROM TAB2);
- ③ SELECT COUNT(\*) FROM TAB1  
WHERE NOT EXISTS (SELECT 1 FROM TAB2 WHERE TAB1.ID = TAB2.ID);
- ④ SELECT COUNT(\*) FROM TAB1 JOIN TAB2  
ON TAB1.ID != TAB2.ID;

문제2. 다음 SQL 중 실행 결과가 다른 하나는?

<TAB1>

| NO | ID  |
|----|-----|
| 1  | AAA |
| 2  | BBB |
| 3  | CCC |
| 4  | DDD |
| 5  |     |

<TAB2>

| NO   | ID  |
|------|-----|
| 1000 | AAA |
| 2000 | CCC |
| 3000 |     |

- ① SELECT COUNT(\*) FROM TAB1  
WHERE NOT EXISTS (SELECT 'X' FROM TAB2 WHERE TAB1.ID = TAB2.ID);
- ② SELECT COUNT(\*) FROM TAB1  
WHERE ID NOT IN (SELECT ID FROM TAB2);
- ③ SELECT COUNT(\*) FROM TAB1  
WHERE NOT EXISTS (SELECT 1 FROM TAB2 WHERE TAB1.ID = TAB2.ID);
- ④ SELECT COUNT(\*)  
FROM (  
SELECT ID  
FROM TAB\_EXISTS1  
WHERE ID IS NOT NULL  
MINUS  
SELECT ID  
FROM TAB\_EXISTS2  
WHERE ID IS NOT NULL  
UNION ALL  
SELECT ID  
FROM TAB\_EXISTS1  
WHERE ID IS NULL  
);



## 유형6. ROLLUP / CUBE / GROUPING SETS 다양한 집합

1. ROLLUP : 오른쪽에서 왼쪽으로 컬럼을 삭제하면서 GROUP BY 진행

- ROLLUP(A,B) : (A,B) + (A) + ()
- ROLLUP(A,B,C) : (A,B,C) + (A,B) + (A) + ()
- A, ROLLUP(B) : (A,B), (A)
- A, ROLLUP(B,C) : (A,B,C), (A,B) + (A)

2. CUBE : 발생 가능한 모든 조합

- CUBE(A,B) : (A,B) + (A) + (B) + ()
- CUBE(A,B,C) : (A,B,C) + (A,B) + (A,C) + (B,C) + (A) + (B) + (C) + ()
- A, CUBE(B) : (A,B), (A)
- A, CUBE(B,C) : (A,B,C), (A,B) + (A,C) + (A)

3. GROUPING SETS : 나열된 조합

- GROUPING SETS (A,B) : (A) + (B)
- GROUPING SETS (A,B,C) : (A) + (B) + (C)
- A, GROUPING SETS (B) : (A,B)
- A, GROUPING SETS (B,C) : (A,B), (A,C)

문제1. 다음 SQL의 실행 결과와 같은 값을 반환하는 식으로, 빈칸에 들어갈 함수식으로 적절한 것은?

```
SELECT PRODUCT_CODE, STORE, SUM(SALES_QTY)
FROM SALES_DATA
GROUP BY PRODUCT_CODE, CUBE(STORE);
```

- ① CUBE(PRODUCT\_CODE, STORE);
- ② ROLLUP(PRODUCT\_CODE, STORE);
- ③ PRODUCT\_CODE, ROLLUP(STORE);
- ④ GROUPING SETS (PRODUCT, ROLLUP(STORE));

## 유형7. GROUPING / GROUPING\_ID

### ● GROUPING / GROUPING\_ID

- ROLLUP, CUBE 등과 같이 다단계 집계를 수행할 때, 각 행이 어떤 집합에 속하는지를 구분하는 함수

#### 1. GROUPING(컬럼1) / GROUPING\_ID(컬럼1)

- 해당 컬럼 값이 집계로 인해 NULL이 된 경우 → 1 반환
- 해당 컬럼 값이 실제 데이터에서 온 경우 → 0 반환

#### 2. GROUPING\_ID(컬럼1, 컬럼2)

- 각 컬럼이 집계로 인해 NULL이면 1, 실제 값이면 0으로 표시하고, 이를 이진수 형태로 결합하여 10진수로 변환한 값을 반환

| GROUPING(컬럼1) | GROUPING(컬럼2) | GROUPING_ID(컬럼1, 컬럼2) |
|---------------|---------------|-----------------------|
| 0             | 0             | 0                     |
| 0             | 1             | 1                     |
| 1             | 0             | 2                     |
| 1             | 1             | 3                     |

예제) EMP에서 DEPTNO별 급여(SAL) 총합을 구한 경우, GROUPING(DEPTNO)와 GROUPING\_ID(DEPTNO)의 출력 결과

```
SQL1 *
1 SELECT DEPTNO, SUM(SAL),
2     GROUPING(DEPTNO) AS GROUP_DEPTNO1,
3     GROUPING_ID(DEPTNO) AS GROUP_DEPTNO2
4 FROM EMP
5 GROUP BY DEPTNO;
```

Result

Grid Result   Server Output   Text Output   Explain Plan   Statistics

|   | DEPTNO | SUM(SAL) | GROUP_DEPTNO1 | GROUP_DEPTNO2 |
|---|--------|----------|---------------|---------------|
| 1 | 20     | 10875    | 0             | 0             |
| 2 | 30     | 9400     | 0             | 0             |
| 3 | 10     | 8750     | 0             | 0             |

👁 GROUPING(DEPTNO)가 0이면 실제 부서 번호가 존재하는 상세 데이터를 가짐

예제) CUBE를 사용한 GROUP BY 결과에서의 GROUPING(DEPTNO), GROUPING(JOB) 값

SQL1 \*

1 ▶

SELECT DEPTNO, JOB, SUM(SAL),

2

GROUPING(DEPTNO), GROUPING(JOB)

3

FROM EMP

4

GROUP BY CUBE(DEPTNO, JOB);

Result

 Grid Result

 Server Output

 Text Output

 Explain Plan

 Statistics

|    | DEPTNO | JOB       | SUM(SAL) | GROUPING(DEPTNO) | GROUPING(JOB) |
|----|--------|-----------|----------|------------------|---------------|
| 1  |        |           | 29025    | 1                | 1             |
| 2  |        | CLERK     | 4150     | 1                | 0             |
| 3  |        | ANALYST   | 6000     | 1                | 0             |
| 4  |        | MANAGER   | 8275     | 1                | 0             |
| 5  |        | SALESMAN  | 5600     | 1                | 0             |
| 6  |        | PRESIDENT | 5000     | 1                | 0             |
| 7  | 10     |           | 8750     | 0                | 1             |
| 8  | 10     | CLERK     | 1300     | 0                | 0             |
| 9  | 10     | MANAGER   | 2450     | 0                | 0             |
| 10 | 10     | PRESIDENT | 5000     | 0                | 0             |
| 11 | 20     |           | 10875    | 0                | 1             |
| 12 | 20     | CLERK     | 1900     | 0                | 0             |
| 13 | 20     | ANALYST   | 6000     | 0                | 0             |
| 14 | 20     | MANAGER   | 2975     | 0                | 0             |
| 15 | 30     |           | 9400     | 0                | 1             |
| 16 | 30     | CLERK     | 950      | 0                | 0             |
| 17 | 30     | MANAGER   | 2850     | 0                | 0             |
| 18 | 30     | SALESMAN  | 5600     | 0                | 0             |

GROUPING(DEPTNO)은 실제 집계과정에서 NULL로 반환되는 행만 1의 값을 가지며 GROUPING\_ID(DEPTNO)도 동일하다.

예제) CUBE를 사용한 GROUP BY 결과에서의 GROUPING(DEPTNO), GROUPING(JOB),  
GROUPING\_ID(DEPTNO, JOB) 값

SQL1 \*

1 ▶

SELECT DEPTNO, JOB, SUM(SAL), GROUPING(DEPTNO), GROUPING(JOB),

2

GROUPING\_ID(DEPTNO, JOB) AS "GROUPING\_ID"

3

FROM EMP

4

GROUP BY CUBE(DEPTNO, JOB);

Result

 Grid Result

 Server Output

 Text Output

 Explain Plan

 Statistics

|    | DEPTNO | JOB       | SUM(SAL) | GROUPING(DEPTNO) | GROUPING(JOB) | GROUPING_ID |
|----|--------|-----------|----------|------------------|---------------|-------------|
| 1  |        |           | 29025    | 1                | 1             | 3           |
| 2  |        | CLERK     | 4150     | 1                | 0             | 2           |
| 3  |        | ANALYST   | 6000     | 1                | 0             | 2           |
| 4  |        | MANAGER   | 8275     | 1                | 0             | 2           |
| 5  |        | SALESMAN  | 5600     | 1                | 0             | 2           |
| 6  |        | PRESIDENT | 5000     | 1                | 0             | 2           |
| 7  | 10     |           | 8750     | 0                | 1             | 1           |
| 8  | 10     | CLERK     | 1300     | 0                | 0             | 0           |
| 9  | 10     | MANAGER   | 2450     | 0                | 0             | 0           |
| 10 | 10     | PRESIDENT | 5000     | 0                | 0             | 0           |
| 11 | 20     |           | 10875    | 0                | 1             | 1           |
| 12 | 20     | CLERK     | 1900     | 0                | 0             | 0           |
| 13 | 20     | ANALYST   | 6000     | 0                | 0             | 0           |
| 14 | 20     | MANAGER   | 2975     | 0                | 0             | 0           |
| 15 | 30     |           | 9400     | 0                | 1             | 1           |
| 16 | 30     | CLERK     | 950      | 0                | 0             | 0           |
| 17 | 30     | MANAGER   | 2850     | 0                | 0             | 0           |
| 18 | 30     | SALESMAN  | 5600     | 0                | 0             | 0           |

문제1. 다음 결과를 포함하는 집합을 얻기 위한 빈칸으로 적절하지 않은 것은?

SELECT PRODUCT\_CODE, STORE, SUM(SALES\_QTY)  
FROM SALES\_DATA  
GROUP BY CUBE(PRODUCT\_CODE, STORE)  
HAVING \_\_\_\_\_;

- ① GROUPING(PRODUCT\_CODE) = 1
- ② GROUPING(STORE) = 1
- ③ GROUPING\_ID(STORE) = 1
- ④ GROUPING\_ID(PRODUCT\_CODE, STORE) = 1

|    | PRODUCT_CODE | STORE | SUM(SALES_QTY) |
|----|--------------|-------|----------------|
| 1  | P004         |       | 554            |
| 2  | P005         |       | 637            |
| 3  | P002         |       | 878            |
| 4  | P001         |       | 775            |
| 5  | P010         |       | 566            |
| 6  | P009         |       | 924            |
| 7  | P008         |       | 902            |
| 8  | P006         |       | 863            |
| 9  | P003         |       | 875            |
| 10 | P007         |       | 622            |

문제2. 다음 결과를 얻기 위한 SQL로 적절한 것은?

|    | DEPTNO | JOB       | SUM(SAL) |
|----|--------|-----------|----------|
| 1  | 10     | CLERK     | 1300     |
| 2  | 10     | MANAGER   | 2450     |
| 3  | 10     | PRESIDENT | 5000     |
| 4  | 10     | 소계        | 8750     |
| 5  | 20     | ANALYST   | 6000     |
| 6  | 20     | CLERK     | 1900     |
| 7  | 20     | MANAGER   | 2975     |
| 8  | 20     | 소계        | 10875    |
| 9  | 30     | CLERK     | 950      |
| 10 | 30     | MANAGER   | 2850     |
| 11 | 30     | SALESMAN  | 5600     |
| 12 | 30     | 소계        | 9400     |
| 13 |        | ANALYST   | 6000     |
| 14 |        | CLERK     | 4150     |
| 15 |        | MANAGER   | 8275     |
| 16 |        | PRESIDENT | 5000     |
| 17 |        | SALESMAN  | 5600     |
| 18 |        |           | 29025    |

- ① SELECT DEPTNO, NVL(JOB, '소계') AS JOB, SUM(SAL)  
FROM EMP  
GROUP BY CUBE(DEPTNO, JOB)  
ORDER BY 1, 2;
- ② SELECT DEPTNO,  
CASE WHEN GROUPING(JOB) = 1 THEN '소계' ELSE JOB END AS JOB,  
SUM(SAL)  
FROM EMP  
GROUP BY CUBE(DEPTNO, JOB)  
ORDER BY 1, 2;
- ③ SELECT DEPTNO,  
CASE WHEN GROUPING(DEPTNO) = 1 AND GROUPING(JOB) = 0 THEN '소계' ELSE JOB END AS  
JOB, SUM(SAL)  
FROM EMP  
GROUP BY CUBE(DEPTNO, JOB)  
ORDER BY 1, 2;
- ④ SELECT DEPTNO,  
CASE WHEN GROUPING(DEPTNO) = 0 AND GROUPING(JOB) = 1 THEN '소계' ELSE JOB END AS  
JOB, SUM(SAL)  
FROM EMP  
GROUP BY CUBE(DEPTNO, JOB)  
ORDER BY 1, 2;